Bash Scripts for Vite

Can we write some bash scripts for executing the main things in my projects?

```
import { defineConfig } from 'vitest/config';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,
    coverage: {
      provider: 'v8',
      reporter: ['text', 'html', 'lcov'],
      include: ['src/**/*.{js,jsx}'],
      exclude: [
        'src/setupTests.js',
        'src/reportWebVitals.jsx',
      ],
      thresholds: {
        lines: 80,
        functions: 80,
        branches: 70,
        statements: 80,
      },
    },
  },
});
```

Yes. For a Vite + React + Vitest project, a simple `scripts/` folder with bash wrappers can make local dev and CI a lot cleaner.

A solid setup would be:

```
project-root/
├── scripts/
│   ├── dev.sh
│   ├── test.sh
│   ├── test-watch.sh
│   ├── coverage.sh
│   ├── build.sh
│   ├── lint.sh
│   └── ci.sh
```

Use these:

`scripts/dev.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail

echo "Starting Vite dev server..."
npm run dev
```

scripts/test.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running Vitest once..."
npx vitest run
```

scripts/test-watch.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running Vitest in watch mode..."
npx vitest
```

scripts/coverage.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running tests with coverage..."
npx vitest run --coverage
```

scripts/build.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Building app..."
npm run build
```

scripts/lint.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running lint..."
npm run lint
```

scripts/ci.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "Installing dependencies..."
npm ci

echo "Running lint..."
npm run lint

echo "Running tests with coverage..."
npx vitest run --coverage

echo "Building app..."
npm run build

echo "CI checks passed."
```

Then make them executable:

```
chmod +x scripts/*.sh
```

And run them like:

```
./scripts/dev.sh
./scripts/test.sh
./scripts/coverage.sh
./scripts/ci.sh
```

Your current Vitest config already supports coverage thresholds, so `coverage.sh` and `ci.sh` will fail automatically if coverage drops below:

- lines: 80
- functions: 80
- branches: 70
- statements: 80

You can also make them a little smarter with a root check:

```bash
#!/usr/bin/env bash
set -euo pipefail

cd "$(dirname "$0")/.."

echo "Running tests with coverage..."
npx vitest run --coverage
```

That way they work no matter where you launch them from.

If you want, I can write you a full polished `scripts/` set plus the matching `package.json` commands for this project.

> Can I have a base script which runs all of them for the main dir

Yeah — what you want is a **single "orchestrator" bash script** that runs everything in your `scripts/` folder from the project root. Think of it like your local CI runner.

Here's a clean, scalable base script 👇

🔥 `scripts/run-all.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail

# Always run from project root
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "🚀 Starting full project pipeline..."

run_step () {
  local name="$1"
  local script="$2"

  echo ""
  echo "================================"
  echo "▶  Running: $name"
  echo "================================"

  if [[ -f "$script" ]]; then
    bash "$script"
    echo "✅ $name passed"
  else
    echo "⚠  Skipping $name (script not found: $script)"
  fi
}

# Ordered execution (edit this order however you want)
run_step "Install Dependencies" "scripts/install.sh"
run_step "Lint" "scripts/lint.sh"
run_step "Unit Tests" "scripts/test.sh"
run_step "Coverage" "scripts/coverage.sh"
run_step "Build" "scripts/build.sh"

echo ""
echo "🎉 All steps completed successfully!"
```

🧠 Optional: Add `install.sh`

If you don't already have it:

`scripts/install.sh`

```bash
#!/usr/bin/env bash
set -euo pipefail

echo "Installing dependencies..."
npm ci
```

✅ Make it executable

```bash
chmod +x scripts/*.sh
```

▶️ Run everything

```bash
./scripts/run-all.sh
```

⚡ Why this is actually powerful (for your setup)

This matches exactly how your **Hydra / CI mindset** works:

- You now have a **local CI pipeline**
- Same script can be reused in **GitHub Actions / GitLab CI**
- Easy to extend:
  - add `e2e.sh`
  - add `deploy.sh`
  - add `typecheck.sh`
- Fails fast (`set -e`) → stops on first issue

🧩 Bonus (very you)

If you want to go a little more "automation brain":

Add parallel execution for non-dependent steps:

```
bash scripts/lint.sh &
bash scripts/test.sh &
wait
```

Or even:

```
PARALLEL=true ./scripts/run-all.sh
```

(we can wire that in if you want)

If you want next level, I can turn this into:

- a **multi-project runner** (runs across multiple repos 👀)
- or plug it directly into your **GitLab CI pipeline YAML**

Just say the word.

```
✅ Install Dependencies passed

==============================
➡️  Running: Lint
==============================
Running lint...
npm error Missing script: "lint"
npm error
npm error Did you mean this?
npm error   npm link # Symlink a package folder
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete lo
```

Yeah this is a simple one — your script is fine, your `package.json` **just doesn't have a** `lint` **script yet**.

Right now your pipeline is doing:

```
npm run lint
```

...but npm is like “that command doesn’t exist.”

✅ Fix Option 1 (Recommended): Add ESLint properly

Install ESLint + React support:

```
npm install -D eslint eslint-plugin-react eslint-plugin-react-hooks eslint-plugin-jsx-a11y
```

Then initialize:

```
npx eslint --init
```

Then add this to your `package.json`:

```
{
  "scripts": {
    "lint": "eslint src --ext .js,.jsx"
  }
}
```

Now your script will work:

```
./scripts/lint.sh
```

⚡ Fix Option 2 (Quick + Minimal): Stub it

If you don’t care about lint yet but want CI to pass:

```
{
  "scripts": {
    "lint": "echo 'No lint configured yet'"
  }
}
```

🧠 Fix Option 3 (Better pipeline behavior)

Make your `run-all.sh` smarter so it **doesn’t fail if a script doesn’t exist**.

Update this part:

```
if [[ -f "$script" ]]; then
  bash "$script"
```

👉 Replace with:

```
if [[ -f "$script" ]]; then
  if bash "$script"; then
    echo "✅ $name passed"
  else
    echo "❌ $name failed"
    exit 1
  fi
else
  echo "⚠ Skipping $name (script not found: $script)"
fi
```

🔥 Even cleaner (detect missing npm script)

Inside `lint.sh`, you can guard it:

```
#!/usr/bin/env bash
set -euo pipefail

if npm run | grep -q "lint"; then
  echo "Running lint..."
  npm run lint
else
  echo "⚠  No lint script found, skipping..."
fi
```

💡 What I'd do (best setup for you)

Given how you build systems:

- Add ESLint (real signal, not noise)
- Keep the **guarded scripts**
- Let `run-all.sh` act like a **mini CI engine**

If you want next level, I can:

- auto-generate your full `.eslintrc`
- wire lint + test + coverage into GitLab CI YAML
- or make this run across **multiple Hydra projects at once**

Just tell me 👍

Pasted text.txt
Document

Yeah this is exactly the kind of dependency hell that happens with ESLint right now — and your logs confirm it.

Let me break down what's actually happening (and fix it cleanly).

💥 What went wrong

From your logs:

- You have **multiple ESLint versions fighting each other**
- `eslint-config-react-app` (from CRA ecosystem) expects ESLint 8-ish
- But your project pulled **ESLint 10** 👇

Found: eslint@10.1.0

Pasted text

That causes:

- peer dependency conflicts
- install failures
- no proper config setup
- AND your `lint` script never got added

🔥 The Clean Fix (do this exactly)

1. Remove broken ESLint setup

```
rm -rf node_modules package-lock.json
npm uninstall eslint eslint-plugin-react eslint-plugin-react-hooks eslint-plugin-jsx-a11y eslint-conf
```

---

2. Install a stable combo (no conflicts)

Use this (this combo actually works with React + Vite):

```
npm install -D eslint@8 eslint-plugin-react eslint-plugin-react-hooks eslint-plugin-jsx-a11y
```

👉 Key move: **force ESLint v8**, not v10

---

3. Create a simple config (skip the broken wizard)

Create:

`.eslintrc.cjs`

```
module.exports = {
  env: {
    browser: true,
    es2021: true,
  },
  extends: [
    'eslint:recommended',
    'plugin:react/recommended',
  ],
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
  },
  plugins: ['react', 'react-hooks'],
  settings: {
    react: {
      version: 'detect',
    },
  },
  rules: {},
};
```

---

4. Add your missing script (THIS is why your bash failed)

In `package.json`:

```
{
  "scripts": {
    "lint": "eslint src --ext .js,.jsx"
  }
}
```

---

5. Reinstall clean

```
npm install
```

---

6. Run it

```
./scripts/lint.sh
```

🧠 Why this fixes everything

- No CRA config → removes dependency chaos
- ESLint v8 → stable ecosystem
- Manual config → no wizard nonsense
- Your bash pipeline now actually has a `lint` target

⚡ Optional (very you)

Upgrade your `lint.sh` to fail CI cleanly:

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running lint..."

npx eslint src --ext .js,.jsx
```

🧬 Real talk (important)

You hit a **classic modern JS trap**:

- ESLint 9/10 ecosystem is still stabilizing
- CRA configs are semi-legacy
- Vite projects should avoid CRA presets entirely

🚀 If you want next level

I can:

- wire ESLint + Vitest + coverage into **GitLab CI YAML**
- add **pre-commit hooks (Husky)** so bad code never commits
- or make your `run-all.sh` behave like a full **Hydra CI runner**

Just say the word 😈

Sources

```
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % ./scripts/lint.sh
Running lint...

> fake-yt-viral-simulator@1.0.0 lint
> eslint src --ext .js,.jsx

Invalid option '--ext' - perhaps you meant '-c'?
You're using eslint.config.js, some command line flags are no longer available. Please see
https://eslint.org/docs/latest/use/command-line-interface for details.
(.venv) liam@Liams-MacBook-Pro yt-viral-sim %
```

Yep — that error means ESLint is using the **new flat config system** (`eslint.config.js`), and with that setup `--ext` is no longer valid. Your logs point to exactly that.

Pasted text

Use this instead.

Update `package.json`:

```
{
  "scripts": {
    "lint": "eslint src"
  }
}
```

Or, if you want to be explicit:

```
{
  "scripts": {
    "lint": "eslint \"src/**/*.{js,jsx}\""
  }
}
```

Then your `scripts/lint.sh` can stay simple:

```
#!/usr/bin/env bash
set -euo pipefail

echo "Running lint..."
npm run lint
```

Why this happened:

- `eslint.config.js` = flat config
- flat config removes some old CLI flags
- `--ext` is one of the removed ones

So the fix is just to let ESLint read the files from the path or glob directly.

If `eslint src` still acts weird, send me your `eslint.config.js` and I'll make it match your Vite + React setup exactly.

Sources

```
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % ./scripts/lint.sh
Running lint...

> fake-yt-viral-simulator@1.0.0 lint
> eslint src


Oops! Something went wrong! :(

ESLint: 8.57.1

Error [ERR_PACKAGE_PATH_NOT_EXPORTED]: Package subpath './config' is not defined by "exports"
in /Users/liam/Development/wkspaces/yt-viral-sim/node_modules/eslint/package.json imported
from /Users/liam/Development/wkspaces/yt-viral-sim/eslint.config.mjs
    at exportsNotFound (node:internal/modules/esm/resolve:313:10)
    at packageExportsResolve (node:internal/modules/esm/resolve:660:9)
    at packageResolve (node:internal/modules/esm/resolve:773:12)
    at moduleResolve (node:internal/modules/esm/resolve:853:18)
    at defaultResolve (node:internal/modules/esm/resolve:983:11)
    at ModuleLoader.defaultResolve (node:internal/modules/esm/loader:801:12)
    at #cachedDefaultResolve (node:internal/modules/esm/loader:725:25)
    at ModuleLoader.resolve (node:internal/modules/esm/loader:708:38)
    at ModuleLoader.getModuleJobForImport (node:internal/modules/esm/loader:309:38)
    at #link (node:internal/modules/esm/module_job:208:49)
```

That error means your config file and your installed ESLint version do not match.

`eslint.config.mjs` is likely using the newer flat-config import pattern like:

```
import { defineConfig } from "eslint/config"
```

But you currently have **ESLint 8.57.1**, and that package subpath is not available there, which is why it crashes. Your logs show ESLint 8.57.1 and the failing `eslint.config.mjs` load.

Pasted text

You have two clean ways to fix it.

Best fix: stay on ESLint 8 and remove flat config

Delete:

```
rm -f eslint.config.js eslint.config.mjs
```

Create `.eslintrc.cjs`:

```
module.exports = {
  root: true,
  env: {
    browser: true,
    es2021: true,
    node: true,
  },
  extends: [
    "eslint:recommended",
    "plugin:react/recommended",
  ],
  parserOptions: {
    ecmaVersion: "latest",
    sourceType: "module",
    ecmaFeatures: {
      jsx: true,
    },
  },
  plugins: ["react", "react-hooks"],
  settings: {
    react: {
      version: "detect",
    },
  },
  rules: {},
};
```

Then set your `package.json` script to:

```
{
  "scripts": {
    "lint": "eslint src --ext .js,.jsx"
  }
}
```

That is the simplest path for your current setup.

Other fix: keep flat config and upgrade ESLint

If you want to keep `eslint.config.mjs`, upgrade to ESLint 9 and align plugins:

```
npm install -D eslint@9 @eslint/js globals eslint-plugin-react eslint-plugin-react-hooks
```

But this is more likely to trigger more package version cleanup, so for your project I would not do this first.

What I recommend

Do this now:

```
rm -f eslint.config.js eslint.config.mjs
```

Then add `.eslintrc.cjs`, keep ESLint 8, and rerun:

```
./scripts/lint.sh
```

If you paste your current `package.json`, I'll give you the exact final versions block and scripts block to drop in.

Sources

Pasted text.txt
Document

Good — this is actually **progress**, not failure.

Your pipeline is now working correctly… it's just exposing real lint issues.
Right now ESLint is basically screaming:

> "Your config doesn't match modern React + Vitest"

Let's fix this cleanly so your CI isn't useless noise.

💥 What's actually wrong

From your output:

1. React errors (TONS of these)

```
'React' must be in scope when using JSX
```

👉 This is **outdated rule** (React 17+ doesn't need `import React`)

---

2. Test errors

```
'describe' is not defined
'it' is not defined
'expect' is not defined
```

👉 ESLint doesn't know about **Vitest globals**

---

3. PropTypes spam

```
'missing in props validation'
```

👉 You're not using PropTypes (totally fine)

---

4. Misc noise

```
no-unused-vars
```

👉 Optional cleanup

✅ THE FIX (this will clean 90% instantly)

Update your `.eslintrc.cjs` to this:

```
module.exports = {
  root: true,
  env: {
    browser: true,
    es2021: true,
    node: true,
  },
  extends: [
    "eslint:recommended",
    "plugin:react/recommended",
  ],
  parserOptions: {
    ecmaVersion: "latest",
    sourceType: "module",
    ecmaFeatures: {
      jsx: true,
    },
  },
  plugins: ["react", "react-hooks"],
  settings: {
    react: {
      version: "detect",
    },
  },

  rules: {
    // ✅ FIX React 17+ JSX issue
    "react/react-in-jsx-scope": "off",

    // ✅ Turn off prop-types (you're not using them)
    "react/prop-types": "off",
  },

  overrides: [
    {
      // ✅ Fix ALL your test file errors
      files: ["**/*.test.{js,jsx}"],
      env: {
        jest: true, // works for vitest globals too
      },
      rules: {
        "no-undef": "off",
      },
    },
  ],
};
```

🧪 Re-run

```
./scripts/lint.sh
```

🔥 What this will do

You will go from:

👉 **324 errors**

Pasted text

to something like:

👉 ~5–20 actual useful warnings/errors

🧠 Why this matters (important for your system mindset)

Right now your lint step is:

❌ blocking CI for irrelevant noise

After this fix it becomes:

✅ actual signal (real code issues only)

That's the difference between:

- useless pipeline
  vs
- **Hydra-grade automated quality gate**

⚡ Optional (next level)

If you want your pipeline to be elite:

1. Auto-fix script

```
npx eslint src --fix
```

2. Add script

```
"lint:fix": "eslint src --fix"
```

---

3. Make CI stricter later

After cleanup:

```
"no-unused-vars": "warn" → "error"
```

🚀 Reality check

You didn't break anything — you just:

- enabled lint
- exposed real config gaps
- now you're aligning tooling with your stack

If you want next step, I can:

- integrate this into your **run-all.sh fail thresholds**
- add **pre-commit hooks**
- or make your lint step **parallelized with tests**

Just say it 😈

Sources

```
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % ./scripts/lint.sh
Running lint...

> fake-yt-viral-simulator@1.0.0 lint
> eslint src --ext .js,.jsx


/Users/liam/Development/wkspaces/yt-viral-sim/src/components/CommentFeed.jsx
  2:10  error  'logEvent' is defined but never used  no-unused-vars

✖ 1 problem (1 error, 0 warnings)

(.venv) liam@Liams-MacBook-Pro yt-viral-sim %
```

Perfect — this is exactly where you want to be.

You went from **324 errors → 1 real issue**. That means your pipeline is now clean and actually useful.

🔍 The remaining error

`'logEvent' is defined but never used`

👉 Location:

`src/components/CommentFeed.jsx`

✅ Fix options (pick one)

Option 1 (best): remove it

If you're not using it:

```
// ❌ remove this
import { logEvent } from '...';
```

Option 2: actually use it

If it’s meant for analytics/logging:

```
logEvent('comment_added');
```

Option 3: intentionally ignore it

If you’re planning to use it later:

```
// eslint-disable-next-line no-unused-vars
import { logEvent } from '...';
```

Option 4 (pipeline-friendly): downgrade rule

If you don't want CI failing over this:

Update `.eslintrc.cjs`:

```
rules: {
  "no-unused-vars": "warn",
}
```

👉 This makes it:

- ⚠️ warning locally
- ✅ doesn't break CI

🧠 My recommendation (for your setup)

Given how you build systems:

- Keep lint **strict for real issues**
- But don't let it block velocity for small stuff

So:

```
"no-unused-vars": "warn"
```

🚀 Where you are now

Your pipeline is officially:

- ✅ Install
- ✅ Lint (working)
- ✅ Tests (Vitest)
- ✅ Coverage
- ✅ Build

That's basically a **full CI system locally**

If you want next step, we can:

- enforce **coverage thresholds in CI fail**
- add **parallel execution (lint + test together)**
- or scale this into a **multi-repo Hydra runner**

You're basically one step away from that 😈

Still getting this:
/Users/liam/Development/wkspaces/yt-viral-sim/src/components/CommentFeed.jsx
  2:10 error 'logEvent' is defined but never used no-unused-vars

✖ 1 problem (1 error, 0 warnings)

Yeah — this one's straightforward now. Your config is working correctly; ESLint is just doing exactly what you told it to do 😄

Right now that rule is still set to **error**, so it fails your script.

🔥 Why it's still failing

Even if you *thought* you changed it, ESLint is still reading:

```
"no-unused-vars": "error"
```

So it blocks execution.

✅ Fix it (guaranteed)

Open your `.eslintrc.cjs` and make sure this is **explicitly there**:

```
rules: {
  "react/react-in-jsx-scope": "off",
  "react/prop-types": "off",

  // 👇 THIS is the fix
  "no-unused-vars": "warn",
},
```

⚠️ Important (common gotcha)

If it's still not working after that, run:

```
rm -rf node_modules/.cache
```

ESLint sometimes caches results.

🧪 Verify rule is applied

Run this:

```
npx eslint src/components/CommentFeed.jsx
```

You should now see:

```
⚠ warning  'logEvent' is defined but never used
```

NOT an error.

💀 If you want it gone instantly

Just delete it:

```
// remove this line
import { logEvent } from '...';
```

🧠 What's happening (important)

You've now reached the point where:

- Your pipeline is **correctly enforcing rules**
- The only failures left are **actual code quality signals**

That's exactly where you want to be.

🚀 Optional (cleanest dev flow)

You can split behavior:

Local dev:

```
"lint": "eslint src"
```

CI (strict):

```
"lint:ci": "eslint src --max-warnings=0"
```

Then:

- `run-all.sh` → uses `lint:ci`
- you locally → use `lint`

If you want, I can:

- auto-tune your rules to match **your coding style**
- or make your pipeline **fail only on critical issues**

You're basically at production-grade tooling now.

Pasted text.txt
Document

my run-all.sh script worked perfectly:

Can we update my ci with these scripts so they work instead of the regular commands there?:
name: Node.js CI

on:

```yaml
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: "npm"

      - run: npm ci
      - run: npm run build --if-present
      - run: npm run coverage
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

Yep — here's your GitHub Actions workflow updated to use your bash scripts instead of the raw npm commands, matching the flow from your successful `run-all.sh` output. Your local run shows install, lint, unit tests, coverage, and build all passing in that order.

Pasted text

```yaml
name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: "npm"

      - name: Make scripts executable
        run: chmod +x scripts/*.sh

      - name: Run full pipeline
        run: ./scripts/run-all.sh
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

If you want to avoid running coverage twice in the future, you could also split your orchestration into two scripts:

- `run-all.sh` for local/full checks
- `ci.sh` for GitHub Actions only

For example, if you want a leaner CI-specific flow:

```yaml
name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    steps:
      - uses: actions/checkout@v4

      - name: Use Node.js ${{ matrix.node-version }}
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: "npm"

      - name: Make scripts executable
        run: chmod +x scripts/*.sh

      - name: Install dependencies
        run: ./scripts/install.sh

      - name: Lint
        run: ./scripts/lint.sh
        env:
          CI: true

      - name: Unit tests
        run: ./scripts/test.sh
        env:
          CI: true

      - name: Coverage
        run: ./scripts/coverage.sh
        env:
          CI: true

      - name: Build
        run: ./scripts/build.sh
        env:
          CI: true

      - name: Upload coverage report
        if: matrix.node-version == '22.x' && success()
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
```

I'd use the first version if you want GitHub Actions to mirror local behavior exactly. The second is better if you want clearer step-by-step logs in the Actions UI.

One thing worth noting from your successful run: tests pass, but `CommentFeed` is still emitting duplicate React key warnings during test execution. That is not failing CI right now, but it is a real issue you may want to clean up next.

Pasted text

Sources

My dev.sh script there is no npm run dev cmd:
(.venv) liam@Liams-MacBook-Pro yt-viral-sim % ./scripts/dev.sh

```
Starting Vite dev server...
npm error Missing script: "dev"
npm error
npm error To see a list of scripts, run:
npm error   npm run
npm error A complete log of this run can be found i

{
  "name": "fake-yt-viral-simulator",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "@vercel/analytics": "^1.5.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-scripts": "5.0.1",
    "recharts": "^3.1.0"
  },
  "scripts": {
    "start": "react-scripts start",
    "build": "react-scripts build",
    "eject": "react-scripts eject",
    "test": "vitest",
    "test:run": "vitest run",
    "coverage": "vitest run --coverage",
    "lint": "eslint src --ext .js,.jsx",
    "lint:fix": "eslint src --fix",
    "lint:ci": "eslint src --max-warnings=0"
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  },
  "devDependencies": {
    "@testing-library/jest-dom": "^6.9.1",
    "@testing-library/react": "^16.3.2",
    "@vitejs/plugin-react": "^6.0.1",
    "@vitest/coverage-v8": "^4.1.1",
    "eslint": "^8.57.1",
    "eslint-plugin-jsx-a11y": "^6.10.2",
    "eslint-plugin-react": "^7.37.5",
    "eslint-plugin-react-hooks": "^7.0.1",
    "jsdom": "^29.0.1",
```

```
    "vitest": "^4.1.1"
  }
}
```

Yep — your project does **not** use `npm run dev`.

Your `package.json` shows this is using `react-scripts`, so the correct local dev command is:

```
npm start
```

because your scripts block has:

```
"start": "react-scripts start"
```

not:

```
"dev": "vite"
```

So update `scripts/dev.sh` to this:

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Starting dev server..."
npm start
```

If you want, you can rename it mentally as “start app” rather than “dev,” but keeping `dev.sh` is fine too.

Your project is a bit hybrid right now:

- `react-scripts` for app start/build
- `vitest` for testing
- `@vitejs/plugin-react` installed, but not actually used for starting the app

So for your current setup, these mappings are the right ones:

```
./scripts/dev.sh        -> npm start
./scripts/build.sh      -> npm run build
./scripts/test.sh       -> npm run test:run
./scripts/coverage.sh   -> npm run coverage
./scripts/lint.sh       -> npm run lint
```

I’d also update `test.sh` so it does not accidentally launch watch mode:

```
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running Vitest once..."
npm run test:run
```

And `test-watch.sh` can be:

```bash
#!/usr/bin/env bash
set -euo pipefail

ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"

echo "Running Vitest in watch mode..."
npm test
```

So the root issue is simple: your script assumed a Vite convention, but your actual app start command is Create React App style.

> Create a readme.md for just the automation scripts here and CI related documentation, basically what to do how to use it all etc. Ensure it can all be copy and pasted from a markdown comment block.

```markdown
# 🚀 Automation Scripts & CI Guide

This document explains how to use the local automation scripts and CI pipeline for this project.

---

# 📂 Project Structure (Relevant)

scripts/
├── dev.sh
├── install.sh
├── lint.sh
├── test.sh
├── test-watch.sh
├── coverage.sh
├── build.sh
└── run-all.sh

.github/workflows/
└── node.yml


---

# ⚙ Overview

This project uses **bash scripts as a local CI system**, which mirrors the behavior of GitHub Actions

### Why this setup?

- ✅ Same commands locally and in CI
- ✅ Faster debugging (no guessing what CI does)
- ✅ Modular scripts (easy to extend)
- ✅ Fail-fast behavior

---

# 🧠 Script Responsibilities

## 🔹 `install.sh`
Installs dependencies cleanly.

npm ci
```

◆ `dev.sh`
Starts the development server.

./scripts/dev.sh

→ Runs:

npm start

◆ `lint.sh`
Runs ESLint on the project.

./scripts/lint.sh

→ Runs:

npm run lint

◆ `test.sh`
Runs tests once (CI-safe).

./scripts/test.sh

→ Runs:

npm run test:run

◆ `test-watch.sh`
Runs tests in watch mode (local dev).

./scripts/test-watch.sh

→ Runs:

npm test

◆ `coverage.sh`
Runs tests with coverage.

./scripts/coverage.sh

→ Runs:

npm run coverage

◆ `build.sh`
Builds the production app.

./scripts/build.sh

```
→ Runs:
```

npm run build

```
---

## 🔥 `run-all.sh` (Main Orchestrator)

Runs the full pipeline:
```

./scripts/run-all.sh

```
### Execution Order:
1. Install dependencies
2. Lint
3. Unit tests
4. Coverage
5. Build

### Behavior:
- Stops on first failure (`set -e`)
- Skips missing scripts safely
- Provides clear output logs

---

# 🪛 Local CI Usage

Run the full pipeline locally:
```

./scripts/run-all.sh

```
This is equivalent to CI and should always pass before pushing.

---

# 🦾 GitHub Actions CI

The CI pipeline uses the same scripts.

## Workflow Summary

- Runs on:
  - push to `main`
  - pull requests to `main`
- Node versions:
  - 20.x
  - 22.x

---

## CI Steps

1. Checkout repo
2. Setup Node
3. Install dependencies
4. Run full pipeline (`run-all.sh`)
5. Upload coverage report

---

## Example Workflow
```

name: Node.js CI

on:
push:
branches: ["main"]
pull_request:
branches: ["main"]

jobs:
build:
runs-on: ubuntu-latest

```
strategy:
  matrix:
    node-version: [20.x, 22.x]

steps:
  - uses: actions/checkout@v4

  - name: Setup Node
    uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}
      cache: npm

  - name: Make scripts executable
    run: chmod +x scripts/*.sh

  - name: Run pipeline
    run: ./scripts/run-all.sh
    env:
      CI: true

  - name: Upload coverage
    if: matrix.node-version == '22.x'
    uses: actions/upload-artifact@v4
    with:
      name: coverage-report
      path: coverage/


---

# ⚠ Important Notes

## React Setup
- Uses `react-scripts`, NOT Vite for dev
- Dev command is:
```

npm start

```
---

## Vitest Setup
- Used instead of Jest
- Test globals handled via ESLint overrides

---

## ESLint Behavior
- React 17+ JSX rule disabled
- PropTypes disabled
- Test files ignore `no-undef`

---

## Common Issues

### ❌ "npm run dev" not found
→ Use:
```

npm start

```
---

### ❌ ESLint errors for test globals
→ Ensure overrides exist for:
```

*.test.js / *.test.jsx

```
---

### ❌ CI fails but local passes
→ Run:
```

./scripts/run-all.sh

```
---

# 🚀 Best Practices

- Always run before pushing:
```

./scripts/run-all.sh

```
- Keep scripts modular (add new ones easily)

- Use:
```

npm run lint:fix

```
to auto-fix lint issues

---

# 🔥 Future Improvements (Optional)

- Parallel execution (lint + test)
- Pre-commit hooks (Husky)
- Multi-repo automation runner
- CI caching optimizations

---

# 🧩 Summary

This setup gives you:

- ✅ Local CI parity with GitHub Actions
- ✅ Clean, modular automation
- ✅ Fast debugging workflow
- ✅ Scalable foundation for bigger systems

---

💡 If it passes `run-all.sh`, it should pass CI.
```

Please redo this but ensure it all stays within one comment block so I can copy and paste it in one click to a markdown file.

🚀 Automation Scripts & CI Guide

This document explains how to use the local automation scripts and CI pipeline for this project.

📂 Project Structure (Relevant)

```
scripts/
├── dev.sh
├── install.sh
├── lint.sh
├── test.sh
├── test-watch.sh
├── coverage.sh
├── build.sh
└── run-all.sh

.github/workflows/
└── node.yml
```

⚙ Overview

This project uses **bash scripts as a local CI system**, which mirrors the behavior of GitHub Actions

Why this setup?

- ✅ Same commands locally and in CI
- ✅ Faster debugging (no guessing what CI does)
- ✅ Modular scripts (easy to extend)
- ✅ Fail-fast behavior

🧠 Script Responsibilities

🔹 install.sh
Installs dependencies cleanly.

```
npm ci
```

🔹 dev.sh
Starts the development server.

```
./scripts/dev.sh
```

Runs:

```
npm start
```

🔹 lint.sh
Runs ESLint on the project.

```
./scripts/lint.sh
```

Runs:

npm run lint

```
---

## 🔹 test.sh
Runs tests once (CI-safe).
```

./scripts/test.sh

```
Runs:
```

npm run test:run

```
---

## 🔹 test-watch.sh
Runs tests in watch mode (local dev).
```

./scripts/test-watch.sh

```
Runs:
```

npm test

```
---

## 🔹 coverage.sh
Runs tests with coverage.
```

./scripts/coverage.sh

```
Runs:
```

npm run coverage

```
---

## 🔹 build.sh
Builds the production app.
```

./scripts/build.sh

```
Runs:
```

npm run build

```
---

## 🔥 run-all.sh (Main Orchestrator)

Runs the full pipeline:
```

./scripts/run-all.sh

```
### Execution Order:
1. Install dependencies
2. Lint
3. Unit tests
4. Coverage
5. Build

### Behavior:
- Stops on first failure (`set -e`)
- Skips missing scripts safely
- Provides clear output logs

---

# 🔪 Local CI Usage

Run the full pipeline locally:
```

./scripts/run-all.sh

```
This is equivalent to CI and should always pass before pushing.

---

# 🦑 GitHub Actions CI

The CI pipeline uses the same scripts.

## Workflow Summary

- Runs on:
  - push to `main`
  - pull requests to `main`
- Node versions:
  - 20.x
  - 22.x

---

## CI Steps

1. Checkout repo
2. Setup Node
3. Make scripts executable
4. Run full pipeline (`run-all.sh`)
5. Upload coverage report

---

## Example Workflow
```

name: Node.js CI

on:
push:
branches: ["main"]
pull_request:
branches: ["main"]

jobs:
build:
runs-on: ubuntu-latest

```
strategy:
  matrix:
    node-version: [20.x, 22.x]

steps:
  - uses: actions/checkout@v4

  - name: Setup Node
    uses: actions/setup-node@v4
    with:
      node-version: ${{ matrix.node-version }}
      cache: npm

  - name: Make scripts executable
    run: chmod +x scripts/*.sh

  - name: Run pipeline
    run: ./scripts/run-all.sh
    env:
      CI: true

  - name: Upload coverage
    if: matrix.node-version == '22.x'
    uses: actions/upload-artifact@v4
    with:
      name: coverage-report
      path: coverage/


---

# ⚠ Important Notes

## React Setup
- Uses `react-scripts`, NOT Vite for dev
- Dev command is:
```

npm start

```
---

## Vitest Setup
- Used instead of Jest
- Test globals handled via ESLint overrides

---

## ESLint Behavior
- React 17+ JSX rule disabled
- PropTypes disabled
- Test files ignore `no-undef`

---

## Common Issues

### ❌ "npm run dev" not found
Use:
```

npm start

```
---

### ❌ ESLint errors for test globals
Ensure overrides exist for:
```

*.test.js / *.test.jsx

```
---

### ❌ CI fails but local passes
Run:
```

./scripts/run-all.sh

```
---

# 🚀 Best Practices

- Always run before pushing:
```

./scripts/run-all.sh

```
- Use auto-fix when needed:
```

npm run lint:fix

```
- Keep scripts modular and extendable

---

# 🧩 Summary

This setup gives you:

- ✅ Local CI parity with GitHub Actions
- ✅ Clean, modular automation
- ✅ Fast debugging workflow
- ✅ Scalable foundation for larger systems

---

💡 If it passes `run-all.sh`, it should pass CI.
```

Redo it and put it all within '''

```
'''
# 🚀 Automation Scripts & CI Guide

This document explains how to use the local automation scripts and CI pipeline for this project.

---

# 📂 Project Structure (Relevant)

scripts/
├── dev.sh
├── install.sh
├── lint.sh
├── test.sh
├── test-watch.sh
├── coverage.sh
├── build.sh
└── run-all.sh

.github/workflows/
└── node.yml

---

# ⚙ Overview

This project uses bash scripts as a local CI system, mirroring GitHub Actions.

Why this setup?

- Same commands locally and in CI
- Faster debugging
- Modular scripts
- Fail-fast behavior

---

# 🧠 Script Responsibilities

## install.sh
Installs dependencies cleanly

Command:
npm ci

---

## dev.sh
Starts the development server

Run:
./scripts/dev.sh

Executes:
npm start

---

## lint.sh
Runs ESLint

Run:
./scripts/lint.sh

Executes:
npm run lint

---

## test.sh
Runs tests once (CI-safe)

Run:
./scripts/test.sh
```

```
Executes:
npm run test:run

---

## test-watch.sh
Runs tests in watch mode

Run:
./scripts/test-watch.sh

Executes:
npm test

---

## coverage.sh
Runs tests with coverage

Run:
./scripts/coverage.sh

Executes:
npm run coverage

---

## build.sh
Builds production app

Run:
./scripts/build.sh

Executes:
npm run build

---

## run-all.sh (Main Orchestrator)

Run:
./scripts/run-all.sh

Execution order:
1. Install dependencies
2. Lint
3. Tests
4. Coverage
5. Build

Behavior:
- Stops on first failure
- Skips missing scripts safely
- Clean output logs

---

# 🔪 Local CI Usage

Run full pipeline locally:

./scripts/run-all.sh

If this passes, CI should pass.

---

# 🦑 GitHub Actions CI

Triggers:
- Push to main
- PRs to main

Node versions:
- 20.x
```

```
- 22.x

---

## CI Steps

1. Checkout repo
2. Setup Node
3. Make scripts executable
4. Run pipeline (run-all.sh)
5. Upload coverage

---

## Example Workflow

name: Node.js CI

on:
  push:
    branches: ["main"]
  pull_request:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    strategy:
      matrix:
        node-version: [20.x, 22.x]

    steps:
      - uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
          cache: npm

      - name: Make scripts executable
        run: chmod +x scripts/*.sh

      - name: Run pipeline
        run: ./scripts/run-all.sh
        env:
          CI: true

      - name: Upload coverage
        if: matrix.node-version == '22.x'
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/

---

# ⚠ Important Notes

React:
- Uses react-scripts (not Vite)
- Start command:
  npm start

Vitest:
- Used instead of Jest
- Globals handled via ESLint overrides

ESLint:
- JSX scope rule disabled
- PropTypes disabled
- Test files ignore no-undef

---
```

```
# ❗ Common Issues

npm run dev not found:
Use:
npm start

ESLint test errors:
Ensure overrides exist for *.test files

CI fails but local passes:
Run:
./scripts/run-all.sh

---

# 🚀 Best Practices

Before pushing:
./scripts/run-all.sh

Auto-fix lint:
npm run lint:fix

Keep scripts modular

---

# 🧩 Summary

You now have:

- Local CI matching GitHub Actions
- Clean automation scripts
- Fast debugging workflow
- Scalable system foundation

If run-all.sh passes, CI should pass.
'''
```